

Project Requirements Document

1. Project Overview

A local, offline AI-powered conversational application designed to support simple spoken conversations. The system will receive English speech, transcribe it into text, generate a short AI response, and convert that response back into artificial speech.

2. Problem Statement

Simple conversational practice can help users develop the ability to initiate and continue conversations. This project aims to provide an AI system that can engage in short, simple, interactive voice-based conversations locally.

3. Project Goals

- Support simple English voice conversations.
- Allow the AI to either initiate or respond to a conversation.
- Generate short, one-line responses.
- Maintain basic conversational context.
- Convert speech to text locally.
- Generate AI responses using a local language model.
- Convert AI-generated text to speech locally.
- Operate offline after all required models are installed.
- Minimize response delay where possible.

4. Target User

The primary user is a child who is developing conversational English skills and can communicate using simple sentences.

5. Core Functional Requirements

- The user can start a conversation with the AI.
- The AI can initiate a conversation with a simple sentence or question.
- The user can provide English speech through a microphone.
- The system transcribes the speech into text.
- The system sends the text and relevant conversation context to a local AI model.
- The AI generates one short and simple response.
- The response is converted into artificial speech.
- The system plays the generated speech to the user.

6. High-Level System Architecture

React Frontend → FastAPI Backend → Speech-to-Text → Conversation Manager → Local LLM → Text-to-Speech → React Frontend

7. Technology Stack

- **Frontend:** React
- **Backend/API:** FastAPI
- **Programming Language:** Python
- **Speech-to-Text:** Local speech recognition model
- **AI Model:** Local LLM
- **Text-to-Speech:** Local TTS model
- **Deployment:** Local computer

8. Conversation Flow

1. User speaks into the microphone.
2. The frontend records the audio.
3. Audio is sent to the FastAPI backend.
4. Speech-to-Text converts the audio into text.
5. The conversation manager prepares the AI input.
6. The local LLM generates a short response.
7. Text-to-Speech converts the response into audio.
8. The frontend receives and plays the audio.
9. The user can continue the conversation.

9. Non-Functional Requirements

- The application should operate locally.
- The application should function offline after installation.
- AI responses should be simple and child-friendly.
- Responses should normally be limited to one sentence.
- The system should provide reasonable response speed.
- The architecture should allow individual AI components to be replaced or improved.

10. Project Limitations

- Child speech may be difficult for Speech-to-Text models to recognize accurately.
- Performance and response speed will depend on local hardware.
- Local language models may produce incorrect or irrelevant responses.
- Fully offline models may provide lower quality than large cloud-based models.
- Speech quality may depend on the selected TTS model.

11. Initial MVP Scope

The first version will include:

- A simple React interface.
- Microphone audio input.
- Local speech transcription.
- Basic conversation initiation.
- One-line AI responses.
- Local text-to-speech output.
- Basic conversation history.

12. Future Enhancements

- Personalized vocabulary and difficulty levels.
- Conversation topics.
- Parent-controlled settings.
- Progress tracking.
- Improved child speech recognition.
- Automatic correction or expansion of user sentences.
- Multiple AI personalities or voices.
- Mobile application support.

13. Success Criteria

The MVP will be considered successful if the user can speak a simple English sentence, receive a relevant short AI response, hear the response through the application, and continue the conversation naturally.